

Lab 17 — The Automotive Bus: CAN (TWAI) — Broadcast by Message ID

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	2–3 hrs
Proves	CAN framing, hardware filtering, bus-off recovery
Prerequisites	Labs 2 and 8.
Beyond the board	Part A: one jumper. Part B: two SN65HVD230 transceivers and two 120 Ω resistors.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§17.1 objective**, **§17.2 parts and wiring**, **§17.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

KEY POINT

How this lab is structured

Concept, then **Phase 0 configure**, **Phase 1 the complete code**, walkthrough, **Phase 2 build and run**, **Phase 3 prove it**, then worked extensions. Every file is given in full. Nothing is undefined and nothing is left for you to work out.

17.1 Objective & TM4C bridge

Send and receive CAN frames. CAN runs cars, industrial machinery and a good deal of aerospace, so it is the bus to know for automotive, industrial and defense work.

TM4C BRIDGE

The TM4C had a CAN controller configured through its own registers. The message-ID-plus-acceptance-filter model is identical; here it is wrapped in setup macros. One naming note: Espressif calls its CAN peripheral **TWAI** (Two-Wire Automotive Interface) for trademark reasons. Every `twai_*` function is CAN.

17.2 Parts & wiring

Part	Needs	Status
A — self-test	Board + one jumper, GPIO5 to GPIO4	Required
B — real bus	2 boards, 2× SN65HVD230, 2× 120 Ω	~\$10

ESP32 (each node)	SN65HVD230	Bus
GPIO5 (TX)	TXD	CAN_H to CAN_H, CAN_L to CAN_L between nodes, with a 120 Ω resistor across H-L at each end
GPIO4 (RX)	RXD	
3V3	VCC	
GND	GND	

WARNING

Buy the 3.3 V transceiver

SN65HVD230 is a 3.3 V part and correct for the ESP32. The very common **MCP2551** is 5 V; its RXD output can exceed the ESP32's 3.3 V limit, and the S3's pins are **not** 5 V tolerant. This is the single most common way people damage a board on a CAN lab.

NOTE

Why you need transceivers at all

The ESP32 produces ordinary 3.3 V logic on TX and RX. Real CAN on the wire is a **differential pair**, CAN_H and CAN_L, driven to specific voltages so that electrical noise picked up by both wires cancels out. That noise immunity is why CAN survives in an engine bay. The transceiver is the chip that converts between logic levels and the differential pair, and it is a physical-layer necessity, not an optional accessory. The 120 Ω resistors at each end match the cable's impedance and prevent signal reflections.

17.3 How it works

CAN runs cars, industrial machinery and a good deal of aerospace, so it is the bus to know for automotive, industrial and defence work. It is also built on a genuinely different idea from every bus so far, and that idea is the interesting part.

First, the naming

Espressif calls its CAN peripheral **TWAI** — Two-Wire Automotive Interface — because "CAN" is a Bosch trademark. Every `twai_*` function in the API is CAN. Knowing that saves confusion when you search the documentation for "CAN" and find nothing.

Nobody has an address

Every bus so far identified a *device*. I²C sends a 7-bit device address; SPI pulls a device's dedicated chip-select line. CAN does neither, and this is the central idea:

A CAN frame carries a message ID that names *what the message is*, not *who it is for*. `0x0C1` might mean "engine RPM." Every node on the bus hears every frame and decides for itself whether it cares. There is no addressing, no destination, and no controller.

The consequences are what make it right for vehicles:

- **Adding a receiver requires changing nothing.** Bolt on a new display that wants engine RPM and no transmitter is modified or even aware.
- **Several nodes can consume the same message** simultaneously — the dashboard, the transmission controller and the logger all read the same RPM frame.
- **There is no controller to fail.** Every node is a peer. Remove any one and the rest keep talking.

Filtering is done in **hardware**: an acceptance filter compares incoming IDs against a mask and only raises an interrupt for matches. On a bus carrying thousands of frames a second, that is what keeps a node's CPU idle instead of drowning.

The physical layer, and why you need transceivers

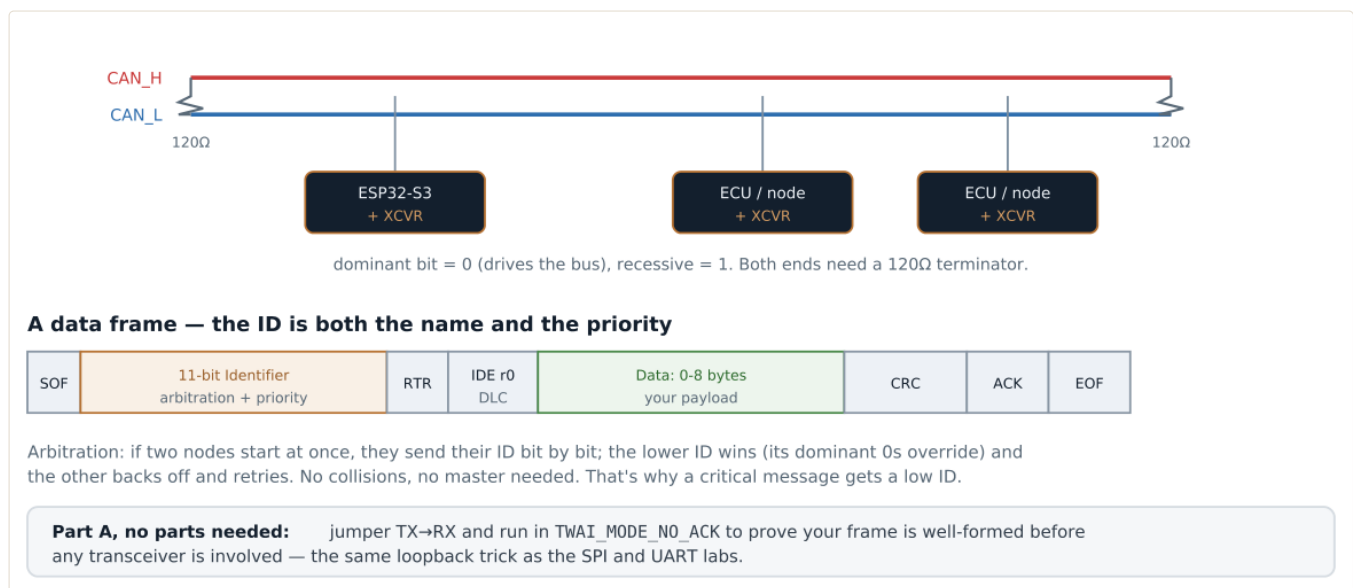


Figure 17.1. Every node taps the same differential pair, terminated at both ends. The 11-bit identifier both names the message and, as the next figure shows, determines its priority.

CAN bus topology and the 2.0A data frame.

Topology: a CAN_H / CAN_L differential pair runs the length of the bus with a 120 ohm terminating resistor at EACH end. Three nodes tap the same pair: the ESP32-S3 plus a transceiver, and two other ECU nodes each with their own transceiver.

Signalling: a dominant bit is 0 and wins the bus; a recessive bit is 1. Every node needs a transceiver, and the bus needs exactly two 120 ohm terminators.

The data frame, field by field: SOF | 11-bit identifier (arbitration = priority) | RTR | IDE/r0/DLC | data, 0 to 8 bytes (your payload) | CRC | ACK | EOF.

Arbitration: if two nodes start at once they transmit their IDs bit by bit; the lower ID wins because its dominant 0s override, and the other backs off and retries. No collisions and no master needed. That is why a safety-critical message is given a low ID.

Part A needs no extra parts: jumper TX to RX and run in `TWAI_MODE_NO_ACK` to prove your frame is well-formed before any transceiver is involved. This is the same loopback trick as the SPI and UART labs.

The ESP32 produces ordinary 3.3 V logic on TX and RX. Real CAN on the wire is a **differential pair**, CAN_H and CAN_L, where the signal is the *difference* between two wires rather than either wire's voltage against ground.

That matters because of where CAN lives. Electrical noise in an engine bay — ignition, alternators, motors — couples into both wires almost equally, so it shifts both voltages together and the *difference* is unchanged. The noise cancels. That is the entire reason CAN survives in environments that would destroy a single-ended signal, and it is why the transceiver is a physical necessity rather than an accessory.

The **120 Ω resistors at each end** match the cable's characteristic impedance. Without them, signals reach the end of the wire and reflect back, interfering with the frames still arriving — the same physics as an unterminated antenna. Two of them, one at each extreme of the bus, not one per node.

WARNING

Buy the 3.3 V transceiver

SN65HVD230 is 3.3 V and correct for the ESP32. The very common **MCP2551** is a 5 V part, and its RXD output can exceed the ESP32's 3.3 V limit — the S3's pins are **not** 5 V tolerant. This is the single most common way people damage a board on a CAN lab, and it is a one-word difference in the part number.

Arbitration: two nodes transmit at once and nothing is lost

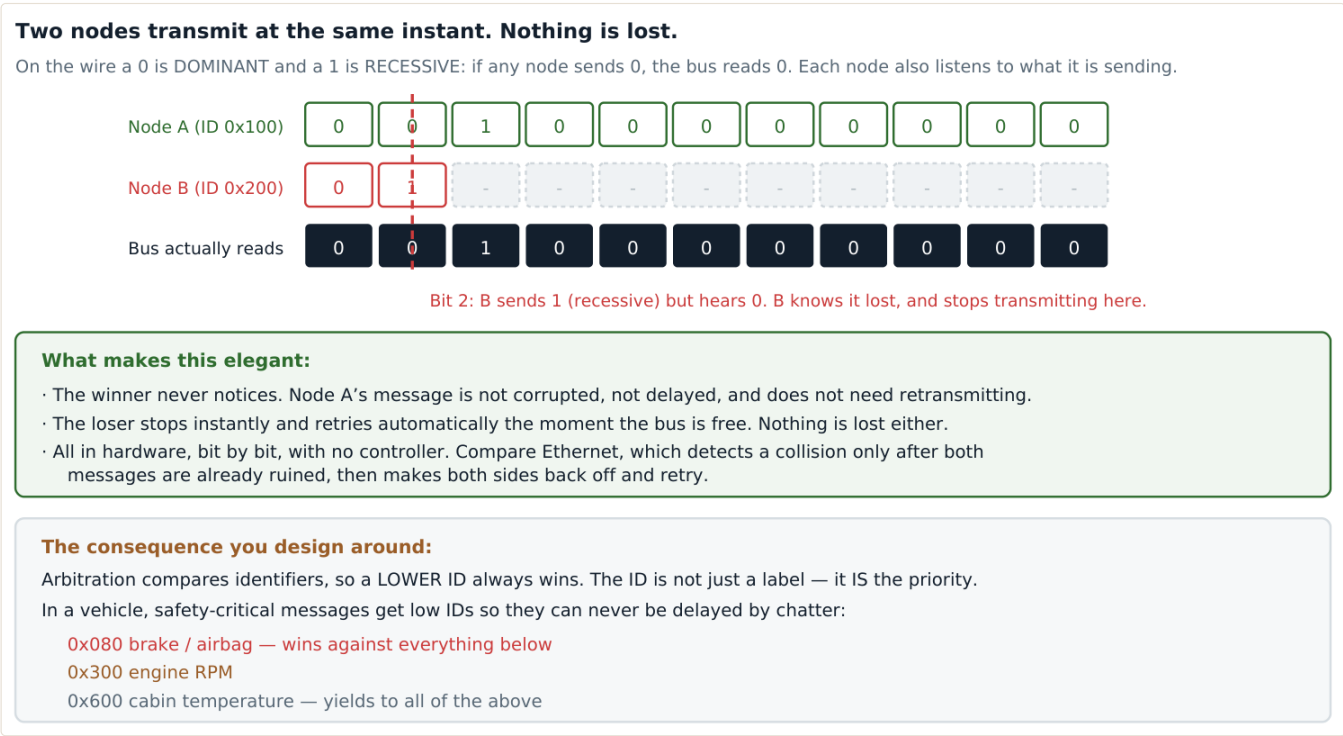


Figure 17.2. Arbitration resolved in hardware, bit by bit, as the identifiers are transmitted. The winner's frame is unaffected and needs no retransmission; the loser detects its loss immediately and retries. The lower identifier always wins, which is why the ID doubles as the priority.

CAN arbitration resolved bit by bit; the lower identifier wins with nothing lost.

On the wire a 0 is DOMINANT and a 1 is RECESSIVE: if any node sends 0, the bus reads 0. Each node also listens to what it is sending.

Two nodes transmit at the same instant, 11-bit identifiers, most significant bit first:

	bit 1	bit 2	bit 3	bit 4	remaining
Node A, ID 0x100	0	0	1	0	0000000
Node B, ID 0x200	0	1	(stops)		
Bus actually reads	0	0	1	0	0000000

At bit 2, Node B sends a recessive 1 but hears a dominant 0. B knows it has lost and stops transmitting immediately.

What makes this elegant: - The winner never notices. Node A's message is not corrupted, not delayed, and does not need retransmitting. - The loser stops instantly and retries automatically the moment the bus is free. Nothing is lost either. - All in hardware, bit by bit, with no controller. Compare Ethernet, which detects a collision only after both messages are already ruined, then makes both sides back off and retry.

The consequence you design around: arbitration compares identifiers, so a LOWER ID always wins. The ID is not just a label, it IS the priority. In a vehicle, safety-critical messages get low IDs so they can never be delayed by chatter: 0x080 brake/airbag wins against everything below; 0x300 engine RPM; 0x600 cabin temperature yields to all of the above.

With no controller, two nodes will eventually start transmitting simultaneously. CAN resolves this in hardware, losslessly, and the mechanism is genuinely elegant.

The bus is wired so that a **0 is dominant** and a **1 is recessive**: if any node drives 0, the bus reads 0, regardless of how many nodes are driving 1. This is the same open-drain, wired-AND arrangement as I²C.

Now, as nodes transmit their identifiers bit by bit, **each one also listens to the bus**. A node that transmits a recessive 1 but hears a dominant 0 knows another node with a lower ID is transmitting, so it immediately stops and becomes a receiver.

The result: the lower ID wins, **the winner's transmission is completely unaffected** — it does not even know a contest happened — and the loser retries automatically as soon as the bus is idle. No data is corrupted and no bandwidth is wasted.

Compare Ethernet's classic approach, where a collision is detected only *after* both messages are already ruined, and both sides then wait a random interval and retry. CAN's scheme resolves contention before any damage occurs, which is what allows you to reason about worst-case latency — the thing a real-time control bus actually needs.

So the ID is the priority. That is a design decision you make when you lay out a system: safety-critical messages get low IDs so nothing can delay them.

Acknowledgement, and why a lone board needs NO_ACK

Every CAN frame must be **acknowledged by at least one other node**. Receivers assert a dominant bit in the ACK slot to confirm they heard it, which gives the transmitter genuine feedback that somebody is listening — something I²C also provides per byte and SPI provides not at all.

It also means a single board with nothing else on the bus will report errors indefinitely, because nobody acknowledges it. After enough consecutive errors the controller takes itself off the bus entirely, a state called **bus-off** — the protocol deliberately removing a node it believes is faulty.

`TWAI_MODE_NO_ACK` waives the acknowledgement requirement so one board can test itself. That is why Part A works, and understanding *why the mode exists* is worth more than the mode itself.

Frames are small on purpose

A CAN frame carries **0 to 8 bytes**. That looks stingy until you consider what it buys: short frames mean short, *bounded* transmission times, which is what lets you calculate the worst-case delay for a high-priority message on a loaded bus. A protocol that allowed large payloads would let one long frame delay a brake message unpredictably.

CAN is a real-time control bus, not a file-transfer link, and every design choice in it trades throughput for predictability. **CAN FD** is the modern extension that allows 64-byte payloads and a faster data phase for applications that need more, and it is worth knowing by name.

17.4 Phase 0 — configure

Three files in the **project root** (the folder holding the top-level `CMakeLists.txt`).

0.1 — sdkconfig.defaults

```
CONFIG_IDF_TARGET="esp32s3"
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
```

That is all. TWAI needs no special Kconfig — unlike Lab 15, there is no hidden switch here that silently disables the feature.

0.2 — top-level CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(lab17_can)
```

0.3 — main/CMakeLists.txt

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS ".")
```

0.4 — apply and verify

```
Remove-Item sdkconfig -ErrorAction SilentlyContinue
idf.py fullclean
idf.py set-target esp32s3
idf.py reconfigure
idf.py build
```

If it builds, Phase 0 is done. There is nothing to grep for in this lab.

0.5 — wiring

Part	Wiring	Mode
A (do this first)	One jumper: GPIO5 to GPIO4 . Nothing else.	<code>TWAI_MODE_NO_ACK</code>
B (optional, needs parts)	Two boards, each with an SN65HVD230 , CAN_H to CAN_H, CAN_L to CAN_L, 120 Ω at each end	<code>TWAI_MODE_NORMAL</code>

WARNING

Buy the 3.3 V transceiver

SN65HVD230 is 3.3 V and correct. The very common **MCP2551** is 5 V, and its RXD output can exceed the ESP32's 3.3 V limit — the S3's pins are **not** 5 V tolerant. This is the most common way people damage a board on a CAN lab, and it is a one-word difference in the part number.

17.5 Phase 1 — the complete main.c

WARNING

Read this before you flash: NO_ACK is not the same as self-reception

These are **two separate settings** and the loopback needs both. Espressif's own self-test example says so explicitly: it uses *"the No Acknowledge Mode and Self Reception Request."*

Setting	Where	What it does
TWAI_MODE_NO_ACK	the <i>driver</i> config	A transmission counts as successful without an ACK from another node. Without this, a lone board goes bus-off in about a second.
.self = 1	each <i>message</i>	Asks the controller to also receive its own transmission . Without this, <code>twai_transmit()</code> succeeds and <code>twai_receive()</code> times out.

Set only the mode — which an earlier edition of this lab did — and you get the confusing symptom of `tx_ok` climbing while `rx_ok` stays at zero, with no error anywhere. `.flags = TWAI_MSG_FLAG_SELF` is the older spelling of the same bit and still works.

```
// =====
// Lab 17 -- CAN / TWAI
//
// Goal:   Send and receive a CAN frame on one board, filter in hardware,
//         watch the error counters, force bus-off and recover from it.
// Verify: rx_ok tracks tx_ok with the jumper fitted. Changing the TX id to
//         OTHER_ID makes rx_ok stop while tx_ok keeps climbing.
// Gotcha: NO_ACK mode and .self = 1 are DIFFERENT things. You need both.
//         In the mask, a 1 bit means DON'T CARE -- inverted from intuition.
//
// Part A: jumper GPIO5 -> GPIO4. No transceiver, no second board.
// =====
#include <stdio.h>
#include <string.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/twai.h"
#include "esp_log.h"

#define TX_PIN      GPIO_NUM_5
#define RX_PIN      GPIO_NUM_4
#define MY_ID       0x123           // the ID this node transmits and accepts
#define OTHER_ID    0x456           // used in proof case 2 to show the filter rejecting

// ---- acceptance filter arithmetic (LEGACY twai_* API, ESP-IDF 5.x) ----
// A single filter with 11-bit standard IDs expects the ID in the TOP 11 bits
// of a 32-bit word, i.e. shifted left by 21:  0x123 -> 0x24600000
//
// In the MASK, a 1 bit means "DON'T CARE". That is inverted from what most
// people assume. The low 21 bits are unused padding below the ID, so they are
// always don't-care. All-11-ID-bits-zero => every ID bit must match exactly.
//
// NOTE for later: ESP-IDF v6's NEW node API reverses this convention -- there
// a mask '1' means "must match". If you ever port this, re-read the docs.
```

```
// -----
#define FILTER_CODE(id) ((uint32_t)(id) << 21)
#define FILTER_MASK_EXACT 0x001FFFFFu

static const char *TAG = "lab17";

static uint32_t g_tx_ok, g_tx_fail, g_rx_ok, g_busoff, g_recovered;

// ----- helpers
static const char *state_name(twai_state_t s)
{
    switch (s) {
        case TWAI_STATE_STOPPED:    return "STOPPED";
        case TWAI_STATE_RUNNING:    return "RUNNING";
        case TWAI_STATE_BUS_OFF:    return "BUS_OFF";
        case TWAI_STATE_RECOVERING: return "RECOVERING";
        default:                    return "?";
    }
}

static void log_status(const char *why)
{
    twai_status_info_t st;
    if (twai_get_status_info(&st) != ESP_OK) return;
    ESP_LOGI(TAG, "[%s] state=%s tx_err=%" PRIu32 " rx_err=%" PRIu32
              " tx_q=%" PRIu32 " rx_q=%" PRIu32,
              why, state_name(st.state),
              st.tx_error_counter, st.rx_error_counter,
              st.msgs_to_tx, st.msgs_to_rx);
}

// ----- TX task
static void tx_task(void *arg)
{
    (void)arg;
    uint8_t counter = 0;

    for (;;) {
        twai_message_t tx = {
            .identifier      = MY_ID,
            .data_length_code = 4,
            .self            = 1,    // <-- SELF RECEPTION. Without this the
            .data             = { 0xDE, 0xAD, 0xBE, counter }, // rx queue stays empty.
        };
        // .extd stays 0 = standard 11-bit ID; .rtr 0 = data frame, not a request

        esp_err_t err = twai_transmit(&tx, pdMS_TO_TICKS(1000));
        if (err == ESP_OK) {
            g_tx_ok++;
        } else {
            g_tx_fail++;
            ESP_LOGW(TAG, "tx failed: %s", esp_err_to_name(err));
        }
        counter++;
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

// ----- RX task
```

```

static void rx_task(void *arg)
{
    (void)arg;
    for (;;) {
        twai_message_t rx;
        esp_err_t err = twai_receive(&rx, pdMS_TO_TICKS(2000));

        if (err == ESP_OK) {
            g_rx_ok++;
            char buf[3 * 8 + 1] = {0};
            for (int i = 0; i < rx.data_length_code && i < 8; i++)
                sprintf(buf + i * 3, "%02X ", rx.data[i]);
            ESP_LOGI(TAG, "RX id=0x%03" PRIx32 " dlc=%d data=[ %s]",
                     rx.identifier, rx.data_length_code, buf);
        } else if (err == ESP_ERR_TIMEOUT) {
            ESP_LOGW(TAG, "no frame in 2 s"); // real information on a bus
        } else {
            ESP_LOGE(TAG, "rx error: %s", esp_err_to_name(err));
        }
    }
}

// ----- alert / recovery
// The task that turns a demo into a node you could ship.
static void alert_task(void *arg)
{
    (void)arg;
    for (;;) {
        uint32_t alerts;
        if (twai_read_alerts(&alerts, pdMS_TO_TICKS(1000)) != ESP_OK) continue;

        if (alerts & TWAI_ALERT_ERR_PASS)
            ESP_LOGW(TAG, "ERROR PASSIVE -- an error counter passed 127");
        if (alerts & TWAI_ALERT_BUS_ERROR)
            ESP_LOGW(TAG, "bus error detected");
        if (alerts & TWAI_ALERT_TX_FAILED)
            ESP_LOGW(TAG, "a transmission failed after retries");
        if (alerts & TWAI_ALERT_RX_QUEUE_FULL)
            ESP_LOGW(TAG, "rx queue full -- frames were dropped");

        if (alerts & TWAI_ALERT_BUS_OFF) {
            g_busoff++;
            ESP_LOGE(TAG, "BUS-OFF. the node removed itself from the bus");
            log_status("bus-off");
            // Recovery MUST be initiated from task context, never an ISR.
            // The controller then waits for 128 occurrences of 11 recessive
            // bits before rejoining -- that is the CAN standard, not IDF.
            vTaskDelay(pdMS_TO_TICKS(100));
            esp_err_t e = twai_initiate_recovery();
            if (e != ESP_OK) ESP_LOGE(TAG, "recovery start failed: %s",
                                     esp_err_to_name(e));
        }

        if (alerts & TWAI_ALERT_BUS_RECOVERED) {
            g_recovered++;
            ESP_LOGW(TAG, "recovered. restarting the driver");
            // Recovery leaves the driver STOPPED, not RUNNING. Miss this and
            // the node sits silent forever, looking like it never recovered.
            esp_err_t e = twai_start();
        }
    }
}

```

```

        if (e != ESP_OK) ESP_LOGE(TAG, "restart failed: %s",
                                   esp_err_to_name(e));
        log_status("recovered");
    }
}

// ----- stats
static void stats_task(void *arg)
{
    (void)arg;
    for (;;) {
        vTaskDelay(pdMS_TO_TICKS(5000));
        ESP_LOGW(TAG, "tx_ok=%" PRIu32 " tx_fail=%" PRIu32 " rx_ok=%" PRIu32
                     " bus_off=%" PRIu32 " recovered=%" PRIu32,
                  g_tx_ok, g_tx_fail, g_rx_ok, g_busoff, g_recovered);
        log_status("periodic");
    }
}

// ----- app_main
void app_main(void)
{
    twai_general_config_t g =
        TWAI_GENERAL_CONFIG_DEFAULT(TX_PIN, RX_PIN, TWAI_MODE_NO_ACK);
    g.alerts_enabled = TWAI_ALERT_ERR_PASS | TWAI_ALERT_BUS_ERROR |
        TWAI_ALERT_TX_FAILED | TWAI_ALERT_RX_QUEUE_FULL |
        TWAI_ALERT_BUS_OFF | TWAI_ALERT_BUS_RECOVERED;
    g.rx_queue_len = 10;
    g.tx_queue_len = 10;

    twai_timing_config_t t = TWAI_TIMING_CONFIG_500KBITS();

    twai_filter_config_t f = {
        .acceptance_code = FILTER_CODE(MY_ID),
        .acceptance_mask = FILTER_MASK_EXACT,
        .single_filter    = true,
    };

    ESP_ERROR_CHECK(twai_driver_install(&g, &t, &f));
    ESP_ERROR_CHECK(twai_start());
    ESP_LOGI(TAG, "TWAI up at 500 kbit/s, accepting only 0x%03X "
                 "(code 0x%08" PRIx32 " , mask 0x%08" PRIx32 ")",
              MY_ID, FILTER_CODE(MY_ID), (uint32_t)FILTER_MASK_EXACT);
    log_status("start");

    // Check every task creation. A silent failure here shows up later as a
    // mystery -- no frames received, or no alerts handled, with no clue why.
    BaseType_t ok = pdPASS;
    ok &= xTaskCreate(rx_task, "rx", 4096, NULL, 6, NULL);
    ok &= xTaskCreate(alert_task, "alert", 4096, NULL, 7, NULL); // above rx
    ok &= xTaskCreate(tx_task, "tx", 4096, NULL, 5, NULL);
    ok &= xTaskCreate(stats_task, "stats", 4096, NULL, 3, NULL);
    if (ok != pdPASS) {
        ESP_LOGE(TAG, "task creation failed -- out of heap?");
        abort();
    }
}

```

17.6 Code walkthrough, line by line

.self = 1 · inside the transmit message

The single line an earlier edition of this lab omitted, and the reason it could not work.

`twai_message_t` holds a union of bitfields and a `flags` word, so `.self = 1` and the older `.flags = TWAI_MSG_FLAG_SELF` set the same bit. It asks the controller to place a copy of the transmitted frame into its own receive queue.

Why the mode alone is not enough: `TWAI_MODE_NO_ACK` changes what counts as a *successful transmission*. Self-reception changes what gets *received*. Two different questions, two different settings. With the jumper fitted but no `.self`, the controller does see its own bits on RX — but a CAN controller normally suppresses reception of a frame it transmitted itself, so the queue stays empty.

#define FILTER_CODE(id) ((uint32_t)(id) << 21)

The bit arithmetic most people get wrong. For a **single filter with 11-bit standard IDs**, the hardware expects the ID in the **top 11 bits** of a 32-bit word, so you shift left by 21. Concretely, `0x123` becomes `0x24600000`.

Get this wrong and the filter appears to work in Part A — because in loopback you only ever hear yourself — and then silently rejects everything on a real bus. Worth checking with a calculator once.

#define FILTER_MASK_EXACT 0x001FFFFF

In the mask, a 1 bit means "don't care." That is inverted from what most people assume, and it is the second half of the same trap.

The low 21 bits are the unused padding below the ID, so they are always don't-care. Setting the top 11 bits to 0 means "every one of these must match." Result: exactly one ID accepted.

To accept a *range*, set the bits that vary. `0x120` through `0x12F` differ only in the bottom four ID bits, so `code = 0x120 << 21` and `mask = (0x00F << 21) | 0x1FFFFF`.

g.alerts_enabled = ...

The line that turns a demo into a node. Alerts are the driver's way of telling you about conditions you would otherwise never see: error-passive, bus errors, failed transmissions, a full receive queue, bus-off, and recovery.

You must opt in. A driver installed with the default config raises none of these, which is why the earlier edition of this lab could not detect bus-off — it never asked to be told.

g.rx_queue_len = 10

How many received frames the driver buffers before dropping. The default is small. On a bus at 500 kbit/s a burst arrives far faster than a task can drain it, so this is your absorption buffer — the same reasoning as the DMA pool depth in Lab 20. `TWAI_ALERT_RX_QUEUE_FULL` tells you when it was not enough.

twai_get_status_info(&st)

The single most useful diagnostic call in the API. It reports the controller **state** and, critically, the two **error counters**.

CAN nodes police themselves with those counters. Every transmit or receive error increments them and every success decrements them. Cross 127 and the node becomes **error passive** — still talking, but no longer allowed to assert dominant error flags. Cross 255 and it goes **bus-off** and removes itself entirely.

That is the protocol deliberately ejecting a node it believes is faulty, so one broken device cannot take down a vehicle bus. Watching those counters climb is the clearest window into bus health you get.

the alert_task, and why recovery lives here

`twai_read_alerts()` blocks until something happens, so this task costs nothing while the bus is healthy.

`twai_initiate_recovery()` **must be called from task context, never from an ISR**. After it is called, the CAN standard requires the controller to observe **128 occurrences of 11 consecutive recessive bits** before rejoining — a mandatory cooling-off period, not an ESP-IDF invention. At 500 kbit/s that is a few milliseconds, and during it the node is deaf.

ESP_ERROR_CHECK(twai_start()) after TWAI_ALERT_BUS_RECOVERED

The step people miss. **Recovery leaves the driver in the STOPPED state, not RUNNING**. If you do not restart it, the node completes recovery and then sits there silently forever, which looks exactly like a node that never recovered at all.

alert_task at priority 7, above rx_task at 6

Deliberate. When the bus is failing, handling the failure matters more than draining frames. If `rx_task` outranked the alert task, a flood of errors could keep you busy receiving while bus-off went unhandled.

esp_err_to_name(err)

Turns an `esp_err_t` into a readable string. Cheap, and it converts `rx error: 0x107` into `rx error: ESP_ERR_TIMEOUT`. Use it in every error log you write.

%03" PRIX32 " for the identifier

`twai_message_t.identifier` is `uint32_t`, whose underlying type varies across ESP-IDF targets and versions, so `%03lX` is not portable and produces a warning on some builds. `PRIX32` from `<inttypes.h>` expands to whatever is correct. Three hex digits is exactly right for an 11-bit ID (max `0x7FF`).

17.7 Phase 2 — build, flash, run

```
idf.py build
idf.py -p COM6 flash monitor
```

KEY POINT

Expected output, Part A with the GPIO5-GPIO4 jumper fitted

```
I lab17: TWAI up at 500 kbit/s, filtering for 0x123
I lab17: [start] state=RUNNING tx_err=0 rx_err=0 tx_q=0 rx_q=0
I lab17: RX id=0x123 dlc=4 data[ DE AD BE 00 ]
```

```
I lab17: RX id=0x123 dlc=4 data=[ DE AD BE 01 ]
I lab17: RX id=0x123 dlc=4 data=[ DE AD BE 02 ]
W lab17: tx_ok=10 tx_fail=0 rx_ok=10 bus_off=0 recovered=0
I lab17: [periodic] state=RUNNING tx_err=0 rx_err=0 tx_q=0 rx_q=0
```

The counter byte incrementing proves these are fresh frames, not one frame echoing.

WARNING

If you see nothing received

Jumper missing or on the wrong pins. GPIO5 to GPIO4, both on the expansion base left column.

Filter wrong. Temporarily swap in `TWAI_FILTER_CONFIG_ACCEPT_ALL()`. If frames appear, your code or mask arithmetic is wrong, not your wiring.

Wrong mode. Without `TWAI_MODE_NO_ACK`, a lone node gets no acknowledgement, error counters climb, and it goes bus-off within a second or two. You would see that in the log now, which is the point of having the alert task.

17.8 Phase 3 — prove it

#	Do this	Expected
1	Run as printed with the jumper.	<code>rx_ok</code> tracks <code>tx_ok</code> , state stays <code>RUNNING</code> , both error counters 0.
2	Prove the filter rejects. Change <code>.identifier</code> in <code>tx_task</code> to <code>OTHER_ID</code> and reflash.	<code>tx_ok</code> still climbs, <code>rx_ok</code> stays 0 , and you get "no frame in 2 s". The hardware dropped every frame without waking the CPU. Change it back.
3	Force bus-off. Change the mode to <code>TWAI_MODE_NORMAL</code> with the jumper still fitted and no transceiver.	Error counters climb, <code>ERROR PASSIVE</code> , then <code>BUS-OFF</code> , then recovery and a restart. <code>bus_off</code> and <code>recovered</code> both increment.
4	Leave case 3 running for two minutes.	It cycles: bus-off, recover, restart, bus-off again. The node never gets permanently stuck — that is the property you are demonstrating.

KEY POINT

Case 3 is the one to record in your debug log

Nothing external is needed. In `NORMAL` mode a node requires acknowledgement from at least one other node; alone, it gets none, so every transmission is an error, the counters climb past 255, and the controller ejects itself.

"I forced a bus-off by transmitting in `NORMAL` mode with no peer, watched the TEC pass 255, caught the alert, and recovered from task context" is a specific, verifiable story that almost no intern candidate has.

17.9 Extensions, worked

NOTE

How numbers in this section are labelled

[DERIVED] arithmetic, shown so you can check it. **[MEASURE]** you must run it and record your own value — no number is printed, because printing one you did not measure is worse than printing none. If you publish results anywhere, only **[MEASURE]** values are yours to claim.

17.9.1 — What the hardware filter is actually saving you

Transmit four different IDs while accepting only one, and count what reaches the CPU. Here is the **complete replacement** for `tx_task` — delete the old one and paste this in its place.

```
static void tx_task(void *arg)
{
    (void)arg;
    static const uint32_t ids[4] = { MY_ID, 0x200, 0x300, OTHER_ID };
    uint8_t counter = 0;

    for (;;) {
        twai_message_t tx = {
            .identifier      = ids[counter & 3],    // rotate through four IDs
            .data_length_code = 4,
            .self            = 1,
            .data            = { 0xDE, 0xAD, 0xBE, counter },
        };
        esp_err_t err = twai_transmit(&tx, pdMS_TO_TICKS(1000));
        if (err == ESP_OK) g_tx_ok++;
        else { g_tx_fail++; ESP_LOGW(TAG, "tx failed: %s", esp_err_to_name(err)); }
        counter++;
        vTaskDelay(pdMS_TO_TICKS(250));
    }
}
```

Nothing else changes. Run for a minute and read the periodic stats line.

KEY POINT

What to expect, and what it means

[DERIVED] One ID in four passes the filter, so `rx_ok` should land near **25% of `tx_ok`**. The other three quarters are dropped by the acceptance filter *in silicon* — no interrupt, no task wake, no CPU cost at all.

[MEASURE] Record your own `tx_ok` and `rx_ok` after 60 s. Then set the filter to `TWAI_FILTER_CONFIG_ACCEPT_ALL()`, rerun, and record both again. The second run should show `rx_ok` equal to `tx_ok`.

Scale it up mentally: a vehicle bus carries hundreds of IDs at thousands of frames per second, and a node that cares about three of them wakes for well under 1% of traffic. **That is why the filter is hardware and not an `if` statement** — the CPU is never disturbed, so it can sleep or do real work.

17.9.2 — Accepting a range of IDs instead of one

Real nodes want a block of IDs. The arithmetic, done for you, for `0x120` through `0x12F`:

```
// 0x120..0x12F differ only in the low 4 bits of the 11-bit ID.
// Mask: 1 = DON'T CARE. Set those 4 ID bits, plus the 21 padding bits.
// code = 0x120 << 21                = 0x24000000
// mask = (0x00F << 21) | 0x001FFFFF = 0x01FFFFFF
twai_filter_config_t f = {
    .acceptance_code = (uint32_t)0x120 << 21,
    .acceptance_mask = ((uint32_t)0x00F << 21) | 0x001FFFFFu,
    .single_filter   = true,
};
```

[MEASURE] Verify by transmitting `0x125` (should be accepted) and `0x135` (should be rejected). Change one line in `tx_task`'s `ids[]` array and watch `rx_ok`.

The general rule: **OR together the bits that differ among the IDs you want, and use any one of them as the code.**

17.9.3 — Proving that a lower ID wins arbitration

WARNING

Be honest about what this needs, and about what receive counts do and do not prove

An earlier edition of this lab suggested counting frames at a "third listener." That needs **three boards and three transceivers**, which you do not have — and worse, **receive counts would not prove arbitration anyway**. A node that loses arbitration retries automatically and normally gets through on a later attempt, so under any sustainable load both IDs arrive in similar numbers. You would measure nothing.

What arbitration actually changes is **latency**, not delivery. So measure latency, or measure arbitration losses directly.

With two boards and two transceivers, the honest single-board-instrumented version is to count **arbitration-lost alerts**, which the controller reports directly:

```
// add to the alerts enabled in app_main:
g.alerts_enabled |= TWAI_ALERT_ARB_LOST;

// and in alert_task:
if (alerts & TWAI_ALERT_ARB_LOST) {
    g_arb_lost++;
    // no logging here in a tight loop -- just count it
}
```

[MEASURE] Run two boards transmitting continuously, one with ID `0x080` and one with `0x600`, both in `TWAI_MODE_NORMAL` on a properly terminated bus. Record `g_arb_lost` on each after 60 s.

[DERIVED] The expectation: the `0x600` node accumulates arbitration losses and the `0x080` node accumulates approximately none, because `0x080` is numerically lower and its dominant bits override. Neither node loses a *frame* — the loser retries — but only one of them is ever delayed.

If you do not have two transceivers, do not fake it. Write the reasoning and the intended method in your debug log, mark it "not run — no second node," and move on. Proof cases 1 through 4 in §17.8 already carry this lab, and an honest "not measured" is worth more than an invented number.

17.10 Docs and references

Topic	Where
TWAI driver API, filters, alerts, recovery	ESP-IDF v5.5 → Peripherals → TWAI
Acceptance filter worked examples	Same page, "Acceptance Filter" section — shows mask derivation for multiple IDs
Bus-off and recovery, runnable	examples/peripherals/twai/twai_alert_and_recovery — deliberately induces errors
Two-node communication	examples/peripherals/twai/twai_network
Error counters and fault confinement	Any CAN 2.0B specification summary; the counter thresholds 127 and 255 are standard, not vendor-specific

NOTE

A note on ESP-IDF v6.0

v6.0 introduces an entirely new node-based TWAI API (`esp_twai_onchip.h`, with `twai_node_*` calls) alongside the legacy one. This lab targets **v5.5** and uses the classic `twai_*` API, which is correct for that version. If you later port this project to v6.0, this is one of the drivers that changed most — which makes it a good one to port.

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

Additional corrections

Format specifier. `twai_message_t.identifier` is `uint32_t`, whose underlying type varies across ESP-IDF targets and versions. `%03lX` is not portable. Use `PRIx32` from `<inttypes.h>`, or cast explicitly.

Framework version note

NOTE

ESP-IDF v6.0 is now the stable release

These labs are written and validated against **ESP-IDF v5.5**, and pinning a version is the right call for reproducibility — every listing here was checked against it. But v5.5 should no longer be described as the current starting point. **ESP-IDF v6.0 has shipped** and carries breaking changes and removals.

Keep building this track on v5.5 so the listings match. Then, as a genuinely useful final exercise, port your integrated project to v6.0 using Espressif's migration guide and record every API and configuration change you had to make. Version migration is ordinary professional work and almost nobody has done it before their first job.

One concrete example of the drift: v6.0 adds `adc_continuous_parse_data()` and `adc_continuous_read_parse()`, which do the per-sample decoding that Lab 20 does by hand.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 17.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17 ←	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.